Ders 8 Genişletilmiş Notu: Final Proje Gösterimleri: Huffman ile Görüntü Sıkıştırma, Aritmetik Kodlama ve Shannon-Fano ile JSON/XML, LZW ve Yarı Dinamik Huffman ile Metin Sıkıştırma

Ders 8 Genişletilmiş Notu: Final Proje Gösterimleri: Huffman ile Görüntü Sıkıştırma, Aritmetik Kodlama ve Shannon-Fano ile JSON/XML, LZW ve Yarı Dinamik Huffman ile Metin Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Düzeni ve Proje Konuları
- 2. Huffman ile Görüntü Sıkıştırma: Ortak Akış
- 3. Kuantalama ile Birlikte Huffman
- 4. Header Tasarımları
- 5. Renk Tablolu Görüntülerde Gri Çıktı Sorunu
- 6. Diğer Görüntü Projeleri ve Sonuçlar
- 7. JSON ve XML Sıkıştırma: Aritmetik Kodlama ve Shannon-Fano
- 8. LZW ile Metin Sıkıştırma
- 9. Yarı Dinamik Huffman ile Bigram Kodlama
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Düzeni ve Proje Konuları

Bu kayıt dönemin son oturumudur ve tamamen dönem projelerinin canlı gösterimine ayrılmıştır. Her öğrenci sırayla ekranını paylaşıp kodunu anlattı, programı çalıştırdı, sıkıştırılmış dosyayı açarak kayıp olmadığını gösterdi. Hoca her gösterimde aşağı yukarı aynı soruları sordu: hangi sıkıştırma yöntemi kullanıldı, header'da ne gönderildi, karşı taraf dosyayı nasıl açıyor, dışarıdan hazır kod kullanıldı mı, sonuç ne oldu. Aşağıda gösterimler konu konu toplanmış, sunanlar "bir öğrenci" diye anılmıştır.

Projelerin konuları şunlardı:

- Renk tablolu (0-255 arası, 8 bitlik) bir görüntünün Huffman kodlamasıyla sıkıştırılması. Gösterimlerin çoğu bu konudaydı.
- JSON ve XML dosyalarının istatistiksel bir yöntemle sıkıştırılması.
- LZW ile metin sıkıştırma.
- Bir metin dosyasının yarı dinamik Huffman ile, yalnız tek karakterler değil ardışık ikili karakterler de (bigram) kodlanarak sıkıştırılması.

Görüntü projesinin metni bir öğrencinin ekranında şöyle görünüyordu (ekrandan, 02:10:00):

```
Verilen bir resim dosyasını (0-255 arası renklerden oluşan - Color Table)
Huffman Kodlama algoritmasını kullanarak sıkıştırınız.
```

- Sıkıştırdığınız dosyayı açan kodu da yazmalısınız.
- İşlem sonucunda bir kayıp olmadığını gösteriniz.

Bir öğrenci de projeyi şöyle özetledi: 0-255 zaten 8 bit demektir; görüntüyü Huffman ile sıkıştırmak, sıkıştırılmış dosyayı çözen metodu yazmak ve algoritmanın kayıpsız olduğunu göstermek istenmektedir.

2. Huffman ile Görüntü Sıkıştırma: Ortak Akış

Görüntü projelerinin hepsi aynı iskeleti izledi. Farklılık header tasarımında, kuantalama yapılıp yapılmadığında ve kayıpsızlığın nasıl gösterildiğindeydi.

2.1 Kodlama adımları

Bir öğrencinin raporundaki adım listesi akışı iyi özetler (ekrandan, 01:30:00):

1. Görüntüdeki piksellerin histogramı çıkarılır (piksel değeri, kaç kez geçtiği).
2. Piksel değerleri sıklığına göre küçükten büyüğe sıralanır.
3. Ardışık toplamlar da bu toplam dizisine "-" önekiyle eklenir; önek, sayının toplama işlemiyle elde edildiğini gösterir.
4. İkili Huffman ağacı oluşturulur.
5. Her piksele karşılık gelen bit dizisi ağaç taranarak elde edilir.
6. Görüntü tek boyuta indirgenir.
7. Piksel dizisindeki her piksele karşılık gelen bit dizisi sonuç dizisine arka arkaya eklenir.

Başka öğrenciler de aynı yolu anlattı: histogram ya da `bincount` ile her değerin kaç kez geçtiği bulunur, gerekirse olasılıklara çevrilir, yaprak düğümleri oluşturulur, en küçük iki düğüm birleştirilerek ağaç kurulur, sonra ağaç kökten yapraklara dolaşılarak kodlar üretilir. Bir öğrenci her düzeyde sol dala 1, sağ dala 0 verdiğini söyledi; hangi dala 0 verildiği bir anlaşmadır, açma tarafında aynı kural kullanıldığı sürece fark etmez.

Kodlanmış görüntü uzun bir 0-1 dizisidir. Bu dizi 8'erli bölünerek baytlara çevrilir ve dosyaya yazılır. Dizinin uzunluğu her zaman 8'in katı olmadığı için sonda 3-4 bit kalabilir; bunlar sıfırla doldurulur (padding). Açmada bu fazlalık sıfırları atabilmek için eklenen bit sayısı (ya da bit dizisinin gerçek uzunluğu) da dosyada saklanır.

2.2 Açma adımları

Açan taraf önce header'dan ağacı ya da kod tablosunu okur. Sonra bit dizisini baştan okuyup ağaçta ilerler; bir yaprağa ulaşınca o piksel değerini çıkışa yazar ve köke döner. Çıkan tek boyutlu dizi, header'daki genişlik ve yükseklik bilgisiyle yeniden iki boyutlu (renkli görüntüde üç boyutlu) matrise çevrilir ve görüntü dosyası olarak yazılır.

Bir öğrencinin kodunda (ekrandan, 00:30:00) açma kısmı şöyleydi: header ve veri `pickle` ile saklanmış, `decompress` çıktısı `np.ubyte` dizisine çevrilip `np.reshape(..., (h, w))` ile matris yapılmış, sonra `Image.fromarray(...)` ile görüntü oluşturulup `convert("P")` ile palet kipine çevrilerek kaydedilmiştir.

2.3 Kayıpsızlığı göstermek

Öğrenciler kayıpsızlığı farklı yollarla gösterdi:

- Orijinal ve açılmış piksel matrislerinin farkını alıp toplamın sıfır olduğunu göstermek ("Difference Between Pixels: 0").
- Açılmış dosyanın boyutunun orijinalle bayt bayt aynı olduğunu göstermek.
- İki dosyayı bir dosya karşılaştırma programıyla karşılaştırmak.
- Açılmış piksel değerlerini bir metin dosyasına yazıp göstermek (ekrandan, 00:50:00: dosyanın büyük kısmı 131 değerinden oluşuyor).

Açılmış görüntüyü çizdirmeyip yalnız piksel matrisini gösteren bir öğrenciye hoca şunu önerdi: girdi bir bitmap dosyası olduğuna göre, bitmap'in başındaki header kısmı olduğu gibi saklanabilir ve açarken bu header ile görüntü yeniden çizdirilebilir. Hatta piksel değerleri ve sütun

sayısı bilindiğine göre, her satırı sırayla basıp sütun sayısına gelince alt satıra geçerek görüntü elde edilebilir.

> **Hocanın vurgusu:** Kayıpsız bir yöntemde açılan dosyanın orijinalle aynı olduğu gösterilmelidir; yalnız boyutların aynı olması yetmez, içerik de aynı olmalıdır.

3. Kuantalama ile Birlikte Huffman

3.1 K-Means ile renk sayısını azaltma

İlk gösterimi yapan öğrenci, renkli görüntüler üzerinde Huffman uygulamasına ek olarak K-Means ile kuantalama ekledi: görüntünün renkleri 32, 64 ya da 128 renge indirgenip bu görüntü de sıkıştırıldı ve iki sonuç karşılaştırıldı. 32 renge düşürülünce bazı renk ayrıntıları kaybolur; örneğin bir kedi görüntüsünde gözdeki maviler kayboluyor, arka plan bulanıklaşıyordu.

Kod tablosu, piksel değeri ile onun kodundan oluşur. 32 renkli görüntünün kod tablosunda en kısa kod 5 bitti; kuantalanmamış görüntüde en kısa kod 7 bitti. Bir palyaço görüntüsünde çok fazla beyaz olduğu için kuantalanmış halde 254 değeri tek bitle gösterildi; hoca 254'ün hangi renk olduğunu sordu, öğrenci beyaza karşılık geldiğini söyledi. Kedi görüntüsünde arka planda siyah fazla olduğu için 0 değeri 2-3 bitle kodlandı.

Sonuç ekranı (ekrandan, 00:13:56):

Ölçü	Değer
Orijinal boyut	25.354.959
Kuantalanmış boyut	25.451.104
Orijinalin sıkıştırılmış boyutu	5.307.683
Kuantalanmışın sıkıştırılmış boyutu	4.326.484
Orijinal sıkıştırma oranı	%79,07
Kuantalanmışın kendine göre oranı	%83,00
Kuantalanmışın orijinale göre oranı	%82,94
İki sıkıştırılmış boyutun farkı	981.199

Kodda oran `100 - (sıkıştırılmış × 100 / orijinal)` ile hesaplanıyor. Kontrol: $1 - 5.307.683/25.354.959 \approx 0{,}7907$; $1-4.326.484/25.354.959 \approx 0{,}8294$; $1-4.326.484/25.451.104 \approx 0{,}8300$ (python ile doğrulandı). Açılmış dosyaların boyutları orijinal ve kuantalanmış dosyalarla aynıydı.

Soru (hoca): K-Means ile kuantalama yapınca açtığında orijinaline nasıl dönebildin? **Cevap:** Dönülemez. Kuantalama bu projede bir tercihtir; görüntüdeki kayıp göze alınıyorsa yapılır. Öğrenci onu yalnız karşılaştırma için koydu: kuantalarsa boyut ne kadar düşer, kuantalamazsa ne olur.

Hoca, kuantalamayla sıkıştırma oranının (sıkıştırılmış/orijinal) düştüğünü, yani daha iyi sıkıştırdığını teyit etti. Öğrenci, farklı renklerin bir arada olduğu görüntülerde kuantalamanın daha çok fark yarattığını, palyaço görüntüsü gibi büyük kısmı beyaz olanlarda ise yalnız birkaç tonun kuantalandığını ve farkın küçük kaldığını söyledi. Kuantalanmış görüntünün çözülmesi de daha kısa sürer, çünkü ağacın seviyesi daha azdır.

3.2 Kuantalama düzeyinin etkisi

Başka bir öğrenci aynı Huffman koduyla farklı kuantalama düzeylerini denedi (derste sözlü, 01:32:10). Kuantalanmamış bir JPEG görüntüde en kısa kod 7, en uzun 12 bit, ortalama 8,5 bit ve kazanç yalnız %3 oldu. Görüntü 16 renge kuantalanınca kazanç %50'ye, 64 renkte %26'ya çıktı. Kuantalanmamış Lena'da kazanç yaklaşık %6 idi. Bir başka öğrenci de Lena'da kuantalama düzeyi 32'den aşağı indirildikçe performansın arttığını gösterdi.

> **Hocanın vurgusu:** Kuantalama yapılmazsa görüntüde çok yüksek sıkıştırma başarısı alınamaz.

3.3 Değerleri en yakın 10'un katına yuvarlama

Bir öğrenci kuantalamayı şöyle yaptı: her piksel değeri hangi 10'un katına daha yakınsa ona yuvarlanır; $12 \to 10$, $18 \to 20$, $161 \to 160$ (birler basamağı 5'ten küçükse aşağı, büyükse yukarı). Hoca yöntemin adını sordu; öğrenci bilmiyordu. Hoca bu yöntemin bir adı olduğunu, aralığı parçalara bölüp değerleri bu parçaların alt ve üst sınırlarına yuvarladığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Bu yöntem büyük olasılıkla düzgün (uniform) skaler kuantalamadır: değer aralığı eşit genişlikte bölmelere ayrılır ve her değer bir temsil değerine eşlenir. Burada bölme genişliği 10'dur; 256 değer yaklaşık 26 düzeye iner.

Kuantalanmış görüntüde 150 değeri en sık geçen değerdi (6.154 kez). Sonuç (ekrandan, 01:50:25): girdi 66.614 bayt, sıkıştırılmış 34.365 bayt, oran (çıktı/girdi) 0,5159, sıkıştırma yüzdesi %48,41. Kontrol: $34.365/66.614 \approx 0{,}5159$, $100 \times (1 - 0{,}5159) \approx 48{,}41$ (python ile doğrulandı). Öğrenci farklı görüntülerde genelde %40-50 arası kazanç aldığını söyledi.

Bu gösterim sırasında hoca, görüntü işleme kitaplarında sürekli geçen üç ünlü test görüntüsünü andı: Lena, bir maymun (şempanze diye anıldı) ve dürbünlü bir adam.

> **Ek bilgi (derste anlatılmadı):** Bu görüntüler büyük olasılıkla Lena, Mandrill (Baboon) ve Cameraman'dır; görüntü işleme ve sıkıştırma literatüründe standart test görüntüleri olarak kullanılırlar.

4. Header Tasarımları

Hocanın her gösterimde sorduğu soru, açan tarafın kodları nereden bildiğiydi. Öğrencilerin tasarımları şöyle ayrıldı.

4.1 Kod tablosu mu, ağaç mı?

Bir öğrenci iki farklı header denedi: birincisinde her değerin karşılığı olan kod doğrudan yazıldı, ikincisinde ağacın kendisi gönderildi. Ağaç gönderildiğinde açma tarafı ağacı yeniden kurmak zorunda olduğu için daha yavaştır, ama daha çok sıkıştırılır. Sonuç (ekrandan, 00:33:50): "Header Compression True" denemesinde orijinal 41.078 bayt, sıkıştırılmış 34.795 bayt, oran 0,8470, kazanç %15,2953. Öğrenciye göre diğer header türünde kazanç yaklaşık %12, oran yaklaşık 0,87 idi. Kontrol: $34.795/41.078 \approx 0{,}8470$, $1 - 0{,}8470 \approx 0{,}1530$ (python ile doğrulandı).

Kod ekranında bu öğrencinin hesapları şu biçimdeydi (ekrandan, 00:33:50):

```
print(
    f" Original File Size : {file_size_org}byte, Compressed File Size :"
    f" {file_size_comp}byte, Compression Ratio : {'%.4f' % (file_size_comp / file_size_org)} "
    f"Compression Performance (%): { '%.4f' % ((1- (file_size_comp / file_size_org)) * 100)}")
if numpy.sum(image - image_decoded) != 0:
    Image.fromarray(image_decoded).save(f'decomp/{image_name}.bmp')
```

Öğrenci ayrıca kurduğu ağacı renkli kenarlarla çizdirdi: kırmızı kenarlar 0, mavi kenarlar 1. Büyük görüntülerde ağaç çok derin olduğundan elle izlemek zordur. Bu görüntüde 0 (siyah) iki bitle gösterilmişti; beyaz 255'tir.

4.2 Ayrıntılı bir header

Bir öğrencinin raporunda gönderilen üst veri (metadata) şöyle tanımlandı (ekrandan, 01:30:00):

Alan	Uzunluk
Genişliğin 255'e tam sayı bölümü	8 bit
Genişliğin 255'e bölümünden kalan	8 bit
Yüksekliğin 255'e tam sayı bölümü	8 bit
Yüksekliğin 255'e bölümünden kalan	8 bit
Header uzunluğunun 255'e tam sayı bölümü	8 bit

Alan	Uzunluk
Header uzunluğunun 255'e bölümünden kalan	8 bit
Kanal sayısı (2, 3, 4)	3 bit
Huffman ağacının LNR bit dizisi	bölüm × 255 + kalan bit
Renk sayısı	9 bit
Renkler	renk sayısı × 8 bit

Raporda gerekçe de yazılıydı: sıkıştırmada kabul edilen bazı üst veriler header olarak gönderilmelidir, aksi halde veri çözülemez. Öğrenci büyük görüntülerde boyut bilgisinin bir bayta sığmayıp programı çökerttiğini, bu yüzden bölüm-kalan gösterimine geçtiğini söyledi. Dört kanallı bir görüntü de programı çökertmiş, kanal alanı bu yüzden 3 bite çıkarılmıştı. Ağaç, LNR (sol-düğüm-sağ, inorder) dolaşmayla bit dizisine çevrilerek gönderilir.

> **Ders sonrası not:** 255'e bölüm ve kalan ile iki bayttan en çok $255 \cdot 255 + 254 = 65.279$ değeri gösterilebilir. 256'ya bölünse (yani 16 bitlik bir sayı) aynı iki baytla 65.535'e kadar gösterilirdi; tasarım büyük olasılıkla 256 yerine 255 kullanmaktadır.

Bu öğrencinin sonuç bölümü (ekrandan, 01:31:40):

- 2, 3 ve 4 kanallı görüntülerde sıkıştırma denenmiştir.
- Çok fazla renk içeren görüntülerde sıkıştırma oranı daha düşüktür.
- Gri seviye, ikili ve az çeşitte renk içeren görüntülerde sıkıştırma oranı oldukça yüksektir.
- Görüntüdeki renkler kuantalanırsa daha iyi bir sıkıştırma elde edilebilir.

Örnek iki boyutlu bir görüntüde: piksel farkı 0, en kısa Huffman kodu 1, en uzun 14, ortalama 7,5 bit, sıkıştırma %61,94.

4.3 Diğer header seçimleri

- Bir öğrenci önce dosya adını (açarken aynı adla açmak için), sonra Huffman ağacını, sonda kalan sıfırları silmek için uzunluk bilgisini, ardından yükseklik ve boyut bilgisini yazdı.
- İlk gösterimdeki öğrenci kod tablosunu (kodlar ve piksel renkleri) gönderdi.
- Bir öğrenci ağacı header'da gönderdi; hoca "ağacı mı gönderdin, frekansları mı, kodları mı?" diye sordu.
- Bir öğrenci dosyayı yazmadan, kodları bir sözlükte bellekte tutup aynı programda çözdü; bu durumda header sorunu ortaya çıkmaz ama gerçek bir sıkıştırıcıda tablo dosyaya yazılmalıdır.
- Bir öğrenci kodları (binary kodları) gönderdiğini söyledi.

  Hocanın vurgusu: Karşı taraf sembol frekanslarını ya da kodları bilmeden dosyayı açamaz. Bu bilgi header'da gönderilmelidir ve header'ın büyüklüğü net kazancı etkiler.

5. Renk Tablolu Görüntülerde Gri Çıktı Sorunu

Birkaç öğrenci aynı sorunla karşılaştı: piksel matrisleri ve dosya boyutları son baytına kadar aynı olduğu halde, açılmış görüntü renkli değil gri görünüyordu. Bir öğrenci 0-255 değerlerini doğru çevirdiğini, ama renk tablosundaki karşılıkları aktaramadığını söyledi.

Hoca önce renk tablosunu açıkladı: elde 256 renkli bir görüntü olabilir; renk tablosunda her renk kodunun karşılığında kırmızı, yeşil ve mavi değerleri vardır. Huffman bu 256 farklı rengin kullanım sıklıklarından kodları oluşturur. Hoca, gri seviyede bir şey çıkmasının sanki 8 yerine 7 bitle işlem yapılıyormuş gibi bir izlenim verdiğini söyledi; öğrenciye bayt olarak tanımlanan bir şeyi başka bir türe çevirip çevirmediğini sordu.

Başka bir öğrenci sorunun header ile ilgili olduğunu, bunun BMP'nin bir özelliği olduğunu söyledi: matristen sıfırdan yeni bir görüntü oluşturulunca kitaplık bunu varsayılan olarak gri seviye kabul edip header'ı ona göre oluşturuyor ve kendi header'ını yazmaya izin vermiyor. Bir başka öğrenci de paleti ve bütün öznitelikleri kopyalamanın işe yaramadığını, ancak orijinal görüntü nesnesini doğrudan kopyalayınca doğru kaydedebildiğini söyledi; bu yüzden piksel farkı sıfırsa orijinal görüntüyü okuyarak kaydetmişti. Sorun derste çözülmedi.

Ek bilgi (derste anlatılmadı): Palet kipindeki (renk tablolu) bir görüntüde piksel değerleri yalnız indistir; renkler ayrı bir palet tablosundadır. İndis matrisinden yeni bir görüntü oluşturulup palet eklenmezse görüntü büyük olasılıkla gri seviye olarak yorumlanır. Çözüm, orijinal dosyanın paletini okuyup yeni görüntüye atamak (Pillow'da `getpalette` ve `putpalette` gibi) ya da paleti de header'da göndermektir.

6. Diğer Görüntü Projeleri ve Sonuçlar

6.1 Çok renkli bir fotoğraf

Bir öğrenci görüntüyü okuyup her piksel değerinin sayısını, sonra olasılıklarını hesapladı, ağacı kurdu ve kodları üretti. Kazanç raporunda şu formülle hesaplandı (ekrandan, 00:40:00):

$$\text{Sıkıştırma Yüzdesi} = \left(1 - \frac{\text{Çıkış Bit Sayısı}}{\text{Giriş Bit Sayısı}}\right) \times 100$$

Giriş bit sayısı	Çıkış bit sayısı	Sıkıştırma yüzdesi
1.784.160	1.611.724	%9,664

Kontrol: $1 - 1.611.724 / 1.784.160 \approx 0{,}09665$ (python ile doğrulandı). Hoca kuantalama yapılmadığı için başarının düşük kaldığını söyledi. Öğrenci renk kanallarını belli ağırlıklarla çarparak piksel değerlerini daha dar bir aralığa çektiğini anlattı.

6.2 Tek değerin baskın olduğu bir görüntü (C ile)

Bir öğrenci projeyi C ile yazdı. 8 bitlik bir BMP okunup genişlik, yükseklik ve piksel değerleri alındı, iki boyutlu piksel dizisi oluşturuldu. Görüntünün büyük kısmı 131 değerindeydi; değerler frekansa göre büyükten küçüğe sıralanıp kodlandı. Ekrandaki kod tablosundan bir kesit (ekrandan, 00:48:50):

```
103 -> 11110
105 -> 11000
112 -> 11111
115 -> 100101
118 -> 11100
122 -> 100001
128 -> 10110
129 -> 11001
130 -> 10100000
131 -> 0
137 -> 100110
```

En yüksek frekanslı 131 değerine tek bitlik `0` kodu verildi; öğrencinin deyişiyle en yüksek frekansa en kısa kod verilerek sıkıştırma yüksek olur. Ortalama kod uzunluğu 1,31 bit, sıkıştırma oranı 0,16, sıkıştırma performansı %83 çıktı. Ağaç olasılıklar toplanarak kuruldu.

Aynı öğrencinin sunumundaki renkli bir BMP için sonuç tablosu (ekrandan, 00:45:14):

Girdi	Sıkıştırma oranı	Sıkıştırma performansı (%)
Renkli.bmp	0,210931	78,906921

Oran ile performans tutarlıdır: $100 \times (1 - 0{,}210931) = 78{,}9069$.

6.3 Gri seviyeli görüntü testi

Hoca bir öğrenciye “siyah beyaz bir resim versem bu algoritma çalışır mı?” diye sordu. Öğrenci denemediğini ama çalışması gerektiğini söyledi; gri seviyeli Lena indirilip PNG’den çevrilmeden verildi ve program açılmış dosyayı üretti. Hoca, öğrencinin denemediği bir şeyi riske girerek denemesini olumlu karşıladı. Aynı öğrencinin renkli görüntüsü yaklaşık 347 KB’tan 61 KB’a inmişti.

Aynı soruyu sonraki öğrenci de kendi kodunda denedi: renkli görüntüde yaklaşık %50, gri seviyeli görüntüde %45 sıkıştırma aldı.

6.4 RGB görüntünün tek boyutlu listeye çevrilmesi

Bir öğrenci internette bulduğu Huffman kodlarının hep metin (string) sıkıştırmak için yazıldığını, bunları renkli görüntüye uyarladığını anlattı. RGB görüntü üç boyutlu bir dizidir; öğrenci iç içe üç `for` döngüsüyle bütün değerleri tek boyutlu bir listeye aldı (ekrandan, 01:40:00):

```
s = []
for x in range(0, len(img)):
    for y in range(0, len(img[x])):
        for z in range(0, len(img[x][y])):
            s.append(img[x][y][z])
print(len(s)*8)
```

Çıktı 3.187.728 bittir; yani görüntüde 398.466 değer vardır. En sık değer 255 olduğundan en kısa kod (`0`) ona verildi; en az kullanılan değerler (ör. 251 ve 220) en uzun kodları aldı. Çözmede bit dizisi önce tek boyutlu listeye, sonra genişlik ve kanal sayısı bilindiği için yeniden üç boyutlu RGB dizisine çevrildi. Öğrenci sıkıştırma performansını yaklaşık %67 buldu.

Hoca bu gösterimde bir dil düzeltmesi yaptı: bu bir “fotoğraf” değil “resim”dir; fotoğraf ayrı bir şeydir.

6.5 Kanal kanal Huffman (MATLAB)

Bir öğrenci MATLAB’da bir arayüz yaptı: görüntü seçilir, Huffman ile kodlanır ve çözülür; arayüz orijinal veri uzunluğunu, kodlanmış veri uzunluğunu, sıkıştırma oranını ve PSNR değerini gösterir (ekrandan, 02:00:00). Kod R, G ve B kanalları için ayrı ayrı çalışır. Çözmeden sonra PSNR 100 gösterildi; öğrenciye göre bu, görüntüde bozulma olmadığı anlamına gelir. Header olarak Huffman sözlüğü veriyle birlikte gönderildi.

> **Ek bilgi (derste anlatılmadı):** PSNR, orijinal ile yeniden oluşturulan görüntü arasındaki ortalama kare hataya dayanır: $\mathrm{PSNR} = 10 \log_{10}(255^2 / \mathrm{MSE})$. Görüntüler aynıysa MSE sıfırdır ve PSNR sonsuz olur; bu yüzden araçlar böyle bir durumda büyük olasılıkla 100 gibi sabit bir üst değer gösterir.

Öğrenci, sıkıştırılmış dosyanın beklenenden büyük çıktığını söyledi. Hoca, Lena’nın sıkıştırılıp açıldığını ve orijinalin elde edildiğini gördüğünü, sorunun ne olduğunu anlamadığını söyledi.

6.6 Farklı ton dağılımlı üç görüntü

Bir öğrenci renkleri çok dağınık, normal ve hemen hep aynı tonda olan üç görüntüyle denedi. En yüksek sıkıştırma, aynı tonların hâkim olduğu üçüncü görüntüde alındı; kazançlar %50 ile %60 arasında değişti. Header’da binary kodlar gönderildi.

6.7 Metin üzerinde Huffman

Farklı bir alandan gelen bir öğrenci Huffman’ı metin üzerinde uyguladı. Önce harflerin histogramını çıkardı (ör. bir harften bir, başka birinden dört tane). Lorem ipsum metniyle yapılan denemede dosya 27.760’tan 14.791’e indi (birim söylenmedi); bu yaklaşık %47 küçülmedir.

7. JSON ve XML Sıkıştırma: Aritmetik Kodlama ve Shannon-Fano

7.1 Aritmetik kodlama ile

Bir öğrenci JSON ve XML dosyalarını aritmetik kodlamayla sıkıştırdı; dosyaya metin gibi baktı. Her karakterin ASCII karşılığının sıklığını çıkardı, sıklıkları kümülatif olarak topladı, sonra her karakter için low ve high değerlerinden yeni low ve high değerlerini hesapladı. Frekans tablosu 256 elemana kadardır; 257. eleman dosya sonu (EOF) sembolüdür.

Açma kodu ekranda şöyleydi (ekrandan, 00:20:00):

```
int[] freq = new int[257];
for (int i = 0; i < 256; i++)
    freq[i] = readInt(in, NUM_BIT);
freq[256] = 1; // EOF symbol

deCompress(new SimpleFrequencyTable(freq), in, out);
```

Soru (hoca): Derste aritmetik kodlamada karakterlerin olasılıklarını çıkarıp hesaplıyorduk. Header'ı nasıl gönderdin? Karşı taraf neye göre açacak? **Cevap:** Öğrenci hatırlayamadı. Ekrandaki kod, açarken 256 frekansın dosyanın başından okunduğunu gösterir; yani frekans tablosu sıkıştırılmış dosyanın başına yazılmıştır.

Bir JSON dosyası 25.529 KB'tan 16.581 KB'a indi ($1 - 16.581/25.529 \approx 0{,}35$, yaklaşık %35 küçülme). Sıkıştırma kayıpsızdır; açılmış dosya orijinalle aynı boyuttaydı.

7.2 Hocanın önerisi: kalıplara kod vermek

Soru (hoca): JSON'da da XML'de de çok tekrar eden kalıplar var; örneğin bir bitiş etiketi ya da `table` gibi bir etiket. Bunları karakter karakter değil de etiketin tamamı olarak sıkıştırmayı düşünmedin mi? **Cevap:** Hayır, metin olarak yaptım.

Hoca bunu açıkladı: XML, JSON gibi dosyalarda sıkıştırmanın farkı buradadır. Karakter bazlı değil, içlerinde tekrar eden, çok sık geçen kalıplar vardır. Bunlar bir kelime gibi düşünülüp her birine bir kod verilir. Aksi halde üç karakterlik bir etiket üç karakterin koduyla kodlanır. Kalıplar yakalansaydı JSON dosyası çok daha iyi sıkıştırılırdı.

Hoca aynı noktayı Shannon-Fano projesinde tekrarladı: JSON, XML ya da HTML dosyalarında hep tekrar eden kalıplar vardır; örneğin HTML'deki bir bağlantı (a href) kalıbı. Bu kalıbın tamamına tek bir kod atanırsa ciddi sıkıştırma elde edilir. Bigram kodlayan metin projesinde de (Bölüm 9) aynı fikri hatırlattı.

> **Hocanın vurgusu:** JSON, XML ve HTML gibi formatlarda tekrar eden kalıplara tek bir kod atamak, karakter bazlı kodlamadan çok daha yüksek sıkıştırma sağlar.

7.3 Shannon-Fano ile

Bir öğrenci aynı konuda Shannon-Fano'yu kendisi kodladı; algoritmasını kendi yazacağı için en hâkim olduğu yöntemi seçtiğini söyledi. Karakter bazında ilerledi: her karakterin frekans listesi çıkarıldı, büyükten küçüğe sıralandı ve kodlar bulundu. Kodların ikili karşılıkları bir dizide tutuldu ve ikili bir dosya olarak yazıldı.

Kodlama döngüsü 8 bitlik bir tampon kullanır (ekrandan, 01:20:00):

```
for char in input_file_content:
    eight_length_code_value += shannon_dictionary_list[ord(char)]
    while len(eight_length_code_value) >= 8:
        binary_value = obtain_binary_representation_8_length_string(eight_length_code_value[:8
        input_file_content_binary_array.append(binary_value.to_bytes(1, 'little'))
        eight_length_code_value = eight_length_code_value[8:]
```

Son karakterden sonra tamponda 8'den az bit kalırsa bunlar başa sıfır eklenerek yazılır ve uzunlukları son eleman olarak saklanır; böylece bilgi kaybolmaz.

Header şeması kodda yorum olarak yazılıydı (ekrandan, 01:20:30):

```
<shannon_dictionary_character_count> + b'\x00' +
[<shannon_dictionary_character> + b'\x00' + <code_block_count> + ...]

b'\x00' -> (directly we keep 0 ascii value)
<shannon_dictionary_character_count> : [how many 255]? [255]? [0-255: how many character]1
<shannon_dictionary_character>       : [how many 255]? [255]? [0-255: how many character]1
<code_block_count> : [1-8]1 (how many bit the count contains)
<code_content>     : [0-255]1 (code's ascii correspond)
```

Öğrencinin anlatımıyla: header'ın ilk alanı sözlükteki karakter sayısıdır; değer 255'ten küçükse doğrudan yazılır, büyükse 255'in katları ayrıca belirtilir. Her ara eleman ayırıcı olarak sıfır baytıdır. Sonra her karakter için önce ASCII karşılığı, sonra ayırıcı, sonra kodun karşılığı gelir. Kod 8 bitten uzunsa (ör. 10 bit) önce ilk 8 bit, sonra kalan 2 bit başına sıfır eklenerek yazılır ve kaç bit olduğu belirtilir. Açarken bu header'dan sözlük yeniden elde edilir.

Sonuçlar: küçük bir XML dosyasında yaklaşık %23, büyük bir JSON dosyasında daha yüksek bir değer alındı. Büyük JSON için ekrandaki karşılaştırma (ekrandan, 01:20:00): ilk dosya 652.309 bayt, diğer dosya 347.775 bayt, "How much compressed" 0,5331, "Compression ratio" 0,4669.

> **Ders sonrası not:** Öğrenci büyük dosyadaki sonucu "%46" diye söyledi. Ekrandaki 0,4669 sıkıştırma oranıdır (sıkıştırılmış/orijinal); boyuttaki küçülme $1 - 347.775/652.309 \approx \%53{,}3$'tür (python ile doğrulandı).

Öğrenciye göre dosya büyüdükçe karakter tekrarları arttığı için başarı yükselir. Açılmış dosya karakter karakter karşılaştırıldığında yalnız en sondaki bir karakter uyuşmuyordu; öğrenci bunu Python'un dosyayı kapatırken sonuna bir satır sonu koymamasına bağladı. Hoca da farkın yalnız son baytta olduğunu not etti.

8. LZW ile Metin Sıkıştırma

8.1 Sözlük boyutu ve küçük metinler

Bir öğrenci LZW ile metin sıkıştırma yaptı. Literatürde İngilizce için sözlüğün hep 256 ile başlatıldığını, Türkçe metinleri de deneyebilmek için bunu büyüttüğünü söyledi. Sözlük karakter tabanlıdır: başta tek karakterleri içerir, sonra büyür. Küçük metinlerde sorun çıktı: büyük ayrılmış sözlük, küçük metne göre çok fazla yer tutuyordu.

Hoca şunu önerdi: sözlüğün yalnız dolduğu kadarı kullanılmalıdır; açılan sözlüğün tamamını karşı tarafa taşımaya gerek yoktur.

Öğrenci iki deneme yaptı: sözlük uzunluğu sınırsız büyüyebilir ya da derste görüldüğü gibi sınırlanır. Sınırlı sözlükte sıkıştırma performansı biraz düştü. Sürekli aynı harfin tekrarlandığı bir metinde en iyi sonuç (yaklaşık %98) alındı; rastgele kelimelerden oluşan bir metinde ise dosya büyük olmasına rağmen sıkışmadı, tersine büyüdü. Hoca, bir derlemden (korpus) alınan dosyaların içinde biçimi farklı metinler (ör. liste dosyaları) de bulunduğunu hatırlattı.

8.2 Sözlüğü 200 ile başlatan bir LZW

Başka bir öğrenci aynı sorunu yaşadığını, sözlüğü 200 elemanla başlattığını anlattı. İlk 200 kod ASCII karakterleridir (ör. `a` = 97); yeni dizgiler 200'den itibaren eklenir: ilk ikili karakter (`aa`) 200'e, sonrakiler 201, 202, … kodlarına konur. Açma tarafında da sözlük boyutu aynı (200) verilmelidir, aksi halde açılamaz; açmada sözlük gönderilmez, aynı mantıkla yeniden kurulur.

Kodlama fonksiyonu ekranda şöyleydi (ekrandan, 01:15:05):

```python
w = ""
result = []
for c in uncompressed:
    wc = w + c
    if wc in dictionary:
        w = wc
    else:
        result.append(dictionary[w])
        # Add wc to the dictionary.
```

```
            dictionary[wc] = dict_size
            dict_size += 1
            w = c
    # Output the code for w.
    if w:
        result.append(dictionary[w])
    return result
```

Örnek çıktı, 30 tane a, bir boşluk, 16 tane b, bir boşluk ve 6 tane a'dan oluşan metin içindir:

```
[97, 200, 201, 202, 203, 204, 205, 200, 32, 98, 209, 210, 211, 212, 98, 32, 204]
```

Öğrenci çıktıyı şöyle okudu: önce a için 97 yazılır, sonra 200 (aa) gelir; 32 boşluk karakteridir; en sondaki 6 tane a 204 koduyla gösterilir. Bu çıktı python ile yeniden üretildi (Örnek Sorular, Soru 4).

Sonuçlar: kodlar dosyaya ondalık metin olarak yazıldığından bir karakter üç karakterle gösterilmiş oldu; bu yüzden küçük dosyalarda dosya büyüdü. Bir milyon basamaklı pi sayısı dosyasında hiçbir tekrar yakalanamadığı için dosya büyüdü. Genel ortalama %65 ile %75 arasında bir kazançtı. Derste görülen "aabaab" tarzı tekrarlı dosya ve büyük dosyalarda benzerlikler çok yakalandığı için kazanç yüksekti.

Hoca sonuçları yorumladı: standart doğal dildeki bir dosya (ör. bir İncil metni) için %75 civarı beklenen değerdir, daha fazlası beklenemez, belki %80'e çıkar. Harf serilerinden oluşan dosyalar özel hazırlanmıştır, normal hayatta öyle dosyalar yoktur. Ne kadar uzun kalıp yakalanırsa o kadar başarılı olunur.

Soru (hoca): Senin de sözlüğün boyu bellekle mi sınırlı? **Cevap:** Evet.

> **Hocanın vurgusu:** LZW'nin başarısı yakalanan kalıpların uzunluğuna bağlıdır; doğal dil metinlerinde %75-80 civarı beklenir.

9. Yarı Dinamik Huffman ile Bigram Kodlama

9.1 Proje ve dosya yapısı

Bir öğrenci bir metin dosyasını yarı dinamik Huffman ile sıkıştırdı. Projenin koşulu yalnız tek karakterleri değil, birbiri arkasına gelen ikili karakter gruplarını (bigram, hocanın deyişiyle n-gram) da kodlamaktı. Sıkıştırılmış dosyanın yapısı (ekrandan, 01:53:30):

```
| 3 bit            | 32 bit              | n bit        | m bit        | 8-((3+32+n+m)%8) bit
| Padding Bit Size | Huffman Tree Length | Huffman Tree | Encoded Text | Padding Bits
```

Dosyanın bayt olarak yazılabilmesi için sonda eksik kalan bitler padding ile tamamlanır; kaç bit eklendiği ilk 3 bitte belirtilir. Ardından gelen 32 bit, Huffman ağacının uzunluğudur. Ağaç LNR (sol-düğüm-sağ) dolaşmayla bit dizisine çevrilmiştir. Arkasından kodlanmış metin gelir.

Soru (hoca): 32 bitten daha fazla olamaz mı? **Cevap:** Şu an bu programda olamıyor. Konuşmada 32 bitlik alanın ağacın kendisi değil ağacın uzunluğu olduğu netleşti.

> **Ders sonrası not:** Ekrandaki $8-((3+32+n+m) \bmod 8)$ formülü, toplam uzunluk 8'in katı olduğunda 0 yerine 8 verir; bu durumda gereksiz bir tam bayt eklenir. 8 değeri 3 bitlik alana da sığmaz (en çok 7). Doğru ifade büyük olasılıkla $(8-((3+32+n+m) \bmod 8)) \bmod 8$'dir.

9.2 Eşik değeriyle bigram seçimi

Soru (hoca): İkili gramları aldın, tek gramları aldın; bunların sayısı çok fazla olabilir. Hepsini ağaca mı yerleştirdin? **Cevap:** Hayır. Ödevde bir eşik belirlenmesi istenmişti; eşik 0-1 arası bir oran olarak verilir. Örneğin 0,2 ile en sık kullanılan bigramların %20'si sembol olarak dahil edilir; eşik değiştirilebilir.

Deneme metinleri bir derlemden alınan iki metin ve bir lorem ipsum metniydi. Shakespeare'in "As You Like It" eseri kodlandığında program her sembolün frekansını ve kodunu listeledi (ekrandan, 01:55:05). Listeden bir kesit:

Sembol	Frekans	Kod
`' '` (boşluk)	19.359	0001
`'e'`	10.380	1111
`'t'`	7.509	01011
`'o'`	7.375	01101
`'a'`	6.032	10101
`'\n'`	4.122	010000
`'e '`	3.043	101000
`'th'`	2.615	111010
`'he'`	2.155	0011100
`'er'`	1.499	1011010
`'an'`	1.352	1101010
`'re'`	1.140	00110000

İstatistikler: orijinal dosya 125.179 bayt, sıkıştırılmış dosya 72.806 bayt, sıkıştırma performansı %41,84. Kontrol: $1 - 72.806/125.179 \approx 0{,}4184$ (python ile doğrulandı). Açılan dosya orijinalle satır satır karşılaştırıldı; bütün satırlar aynıydı.

Hoca bu listeyi sınıfa gösterdi: `an`, `er` gibi ikili semboller, satır sonu ile sekme gibi ikili karakterler, `th` gibi gruplar ayrı birer kod almıştır. JSON, HTML ve XML sıkıştıranlar da kalıplara böyle bir kod atayabilseydi elde ettikleri sonuçların üstünde bir başarı alırlardı.

Soru (hoca): Toplam kaç sembolün var ve bunların kaç tanesi iki ya da daha fazla karakterden oluşuyor? Ağacın kaç yaprağı var, yaprakların yüzde kaçı çok karakterli? **Cevap:** Öğrenci sayıyı tutmuyordu. Tahminini şöyle yaptı: tek karakter sayısı 30 ise yan yana gelebilecek 900 bigram olur; bunların çoğu metinde vardır. Program yaklaşık 780 sembol gösterdi.

Hoca bu çalışmayı değişik bulduğunu söyledi.

Örnek Sorular

Soru 1 (derste sözlü, 00:06:31): Huffman'dan önce K-Means ile renk sayısı azaltılmış bir görüntü açıldığında orijinal görüntü elde edilebilir mi?

> **Çözüm:** Hayır. Kuantalama kayıplı bir adımdır; Huffman kodlaması kuantalanmış görüntüyü kayıpsız geri verir, ama kuantalamada kaybolan renk ayrıntıları geri gelmez. Kuantalama, kayıp göze alınıyorsa yapılan isteğe bağlı bir adımdır.

Soru 2 (derste sözlü, 00:19:24): Aritmetik kodlamada (ya da Huffman'da) sıkıştırılmış dosyayı açan taraf sembol olasılıklarını nereden bilir?

> **Çözüm:** Kodlayıcı frekans tablosunu (ya da kodları, ağacı) header'a yazar; çözücü aynı tabloyu okuyup aynı aralıkları ya da aynı ağacı kurar. Gösterilen kodda açma fonksiyonu 256 frekansı dosyanın başından okuyup EOF sembolüne frekans 1 vermektedir (ekrandan, 00:20:00).

Soru 3 (derste sözlü, 00:17:17): JSON ve XML dosyalarını karakter bazlı sıkıştırmak yerine nasıl daha iyi sıkıştırırsınız?

> **Çözüm:** Bu dosyalarda sık tekrar eden etiket ve kalıplar vardır. Her kalıp bir kelime gibi düşünülüp tek bir kodla gösterilirse, birkaç karakterlik bir etiket birkaç karakter kodu yerine tek kodla yazılır ve sıkıştırma çok artar.

Soru 4 (ekrandan, 01:15:05): Sözlüğü ilk 200 ASCII koduyla başlatılan, yeni girdileri 200'den itibaren ekleyen LZW ile 30 tane `a`, bir boşluk, 16 tane `b`, bir boşluk ve 6 tane `a`'dan oluşan metni kodlayın.

> **Çözüm:** Çıktı: 97, 200, 201, 202, 203, 204, 205, 200, 32, 98, 209, 210, 211, 212, 98, 32, 204. Sözlüğe sırayla `aa` (200), `aaa` (201), `aaaa` (202), `aaaaa` (203), `aaaaaa` (204), `aaaaaaa` (205), `aaaaaaaa` (206) eklenir; 30 tane `a`, 1 + 2 + 3 + 4 + 5 + 6 + 7 + 2 olarak 97, 200, …, 205, 200 kodlarıyla çıkar. Sondaki 6 tane `a` tek kodla (204) gösterilir. Ekrandaki çıktı python ile yeniden üretildi.

Soru 5 (ekrandan, 00:40:00): Giriş 1.784.160 bit, çıkış 1.611.724 bittir. Sıkıştırma yüzdesini $(1 - \text{çıkış}/\text{giriş}) \times 100$ ile hesaplayın.

> **Çözüm:** $1.611.724/1.784.160 \approx 0{,}9034$; $(1 - 0{,}9034) \times 100 \approx \%9{,}66$. Ekrandaki değer %9,664'tür. Hoca, kuantalama yapılmadığı için bu değerin düşük kaldığını söyledi.

Soru 6 (ekrandan, 01:55:05): Bigram kodlamalı yarı dinamik Huffman ile 125.179 baytlık dosya 72.806 bayta inmiştir. Sıkıştırma oranı ve sıkıştırma performansı nedir?

> **Çözüm (derste verilmedi):** Oran $= 72.806/125.179 \approx 0{,}5816$; performans $= (1 - 0{,}5816) \times 100 \approx \%41{,}84$ (python ile doğrulandı). Ekranda yalnız performans (%41,84) yazıyordu.

Soru 7 (derste sözlü, 01:00:52): Renkli görüntü için yazılmış bir Huffman sıkıştırıcısına siyah beyaz (gri seviyeli) bir görüntü verilirse çalışır mı?

> **Çözüm:** Derste denendi ve çalıştı. Huffman yalnız sembol frekanslarına bakar; gri seviyeli görüntüde tek kanal ve en çok 256 sembol vardır. Bir öğrencinin kodunda gri seviyeli görüntüde %45, renkli görüntüde yaklaşık %50 kazanç alındı.

Soru 8 (derste sözlü, 01:48:37): Piksel değerlerini en yakın 10'un katına yuvarlayan kuantalama (12 → 10, 18 → 20) nasıl bir yöntemdir?

> **Çözüm (derste verilmedi):** Hoca adını hatırlayamadı, yöntemi aralığı parçalara bölüp alt ve üst sınırlara yuvarlamak diye tarif etti. Bu, adım genişliği 10 olan düzgün skaler kuantalamadır: $q(x) = 10 \cdot \text{round}(x/10)$. Farklı değer sayısı 256'dan yaklaşık 26'ya iner, Huffman ağacı küçülür, ama işlem kayıplıdır.

Soru 9 (derste sözlü, 01:56:06): Tek karakterlerin yanında bigramlar da sembol yapılırsa ağaçta kaç yaprak olabilir?

> **Çözüm:** Derste öğrencinin tahmini: 30 tek karakter varsa en çok $30 \times 30 = 900$ bigram olabilir; eşik değeri yalnız en sıklarının alınmasını sağlar. Gösterilen programda yaklaşık 780 sembol vardı.

Soru 10 (derste sözlü, 00:24:24): Renk tablolu bir görüntü Huffman ile sıkıştırılıp açıldığında piksel matrisleri aynı olduğu halde görüntü gri çıkıyor. Neden olabilir?

> **Çözüm:** Derste kesin neden bulunmadı. Öğrenciler sorunun header ile ilgili olduğunu, matristen yeni oluşturulan görüntünün varsayılan olarak gri seviye kabul edildiğini söyledi. Piksel değerleri renk tablosundaki indislerdir; renk tablosu (palet) yeni görüntüye aktarılmazsa indisler gri seviye olarak gösterilir (Bölüm 5'teki ek bilgi).

Dersin Toparlaması

Bu son oturumda dönem projeleri gösterildi. Görüntü projelerinin hepsi aynı Huffman iskeletini kullandı: histogram, sıralama, ağaç, kodlar, bit dizisi ve padding; açma tarafında header'dan ağaç ya da kod tablosu okunup görüntü yeniden kuruldu. Sonuçlar görüntünün renk dağılımına çok bağlıydı: tek bir değerin baskın olduğu görüntülerde ortalama kod uzunluğu 1,3 bite inip %83 kazanç alındı, çok renkli fotoğraflarda kazanç %3-10'da kaldı. K-Means ya da 10'un katına yuvarlama ile yapılan kuantalama kazancı belirgin biçimde artırdı, ama kayıplıdır. Header tasarımında kod tablosu ile ağacın gönderilmesi karşılaştırıldı, büyük sayılar için bölüm-kalan gösterimi kullanıldı. Renk tablolu görüntülerde paletin aktarılamaması yüzünden açılan görüntünün gri çıkması ortak bir sorundu.

Metin tarafında aritmetik kodlama ve Shannon-Fano ile JSON/XML, LZW ile düz metin ve bigramlı yarı dinamik Huffman ile edebi metin sıkıştırıldı. Hocanın dersten çıkardığı ana ders şuydu: JSON, XML ve HTML gibi yapılandırılmış metinlerde karakter yerine tekrar eden kalıplara kod vermek gerekir; bigram kodlaması bu fikrin küçük bir örneğidir. LZW'de kazanç yakalanan kalıpların uzunluğuna bağlıdır; doğal dil metinlerinde %75-80 civarı beklenirken tekrarsız veri dosyayı büyütebilir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.